
Telaria

Aplicación móvil colaborativa
para a xestión integral de viaxes en grupo

Jorge Otero Pailos

Titores: Víctor J. Gallego Fontenla · Pedro Gamallo Fernández

Grao en Enxeñaría Informática — ETSE · USC — Xuño 2026



«Avisaches a Laura?»
«Pásasme as fotos?»



«É hoxe este plan...
ou o outro?»

Organizar unha viaxe en grupo é un caos disperso

Cada parte organizase por separado e a información acaba **espallada** —e ás veces **pérdese**— entre apps distintas:

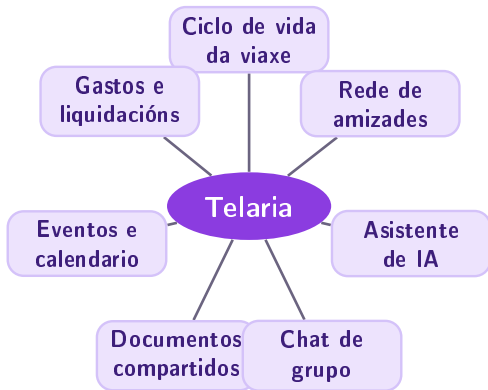


«Onde están os billetes?»



«Quen pagou a comida?
A quen lle debo cartos?»

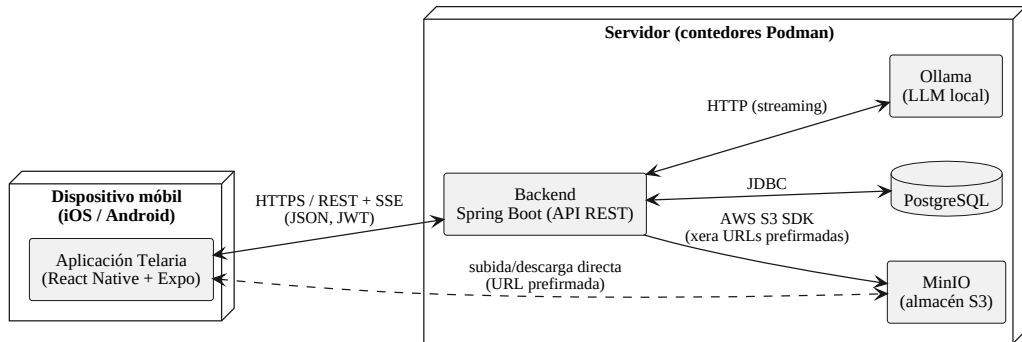
Telaria integra nun só lugar todo o que precisa unha viaxe en grupo, aforrando esforzo e previndo erros:



Ademais

- Todo **nunha soa aplicación**, pensada para viaxes en grupo
- **Sen “organizador”**: todos os membros teñen as mesmas capacidades, evitando atrancos e dependencias dunha soa persoa

Obxectivo: unha app móbil multiplataforma que resolva a fragmentación nun único espazo colaborativo.



Ciente-servidor, porque hai datos compartidos en tempo real (gastos, eventos, chat) que deben estar sincronizados e consistentes para todos.

Ciente: app móbil multiplataforma (React Native + Expo).

Servidor: Spring Boot + (contedores) PostgreSQL + MinIO (S3) + Ollama.

Enfoque **API-first**: OpenAPI é o contrato que se escribe primeiro.

Tecnoloxía	Por que a escollemos
React Native + Expo	Cliente iOS e Android cunha soa base de código TypeScript; Expo simplifica o <i>build</i> e o acceso ás APIs nativas.
Spring Boot	Ecosistema Java maduro: inxección de dependencias, seguridade e integración natural coa xeración de código OpenAPI .
PostgreSQL	Base relacional ACID , idónea para datos moi relacionados (viaxes, membros, gastos) con integridade referencial.
MinIO	Almacén de obxectos compatible con S3 e autohospedable: documentos fóra da BD e migración a S3 sen tocar código .
Ollama	LLM en local : privacidade, sen custos de APIs externas e API REST estándar que serve calquera modelo.



Por que

- O contrato OpenAPI escríbese **primeiro**: única fonte de verdade.
- A partir del **xérase código** nos dous extremos: interfaces e DTOs no backend, tipos TypeScript no frontend.
- Calquera incoherencia salta en **tempo de compilación**, non en produción.
- Ademais: documentación viva (Swagger UI).

Access token · JWT

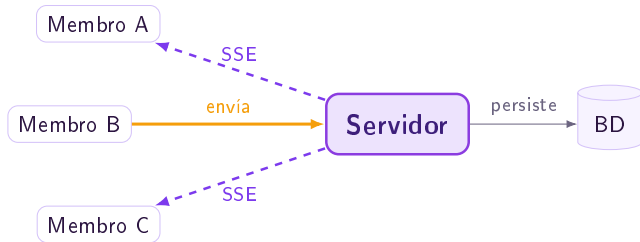
- En **cada petición** (moi frecuente)
- Validado pola **sinatura** RSA
- **Sen tocar a BD** → *stateless*

Refresh token · opaco

- Só **ao renovar** (pouco frecuente)
- Cadea aleatoria **gardada na BD**
- Por iso **pódese invalidar**

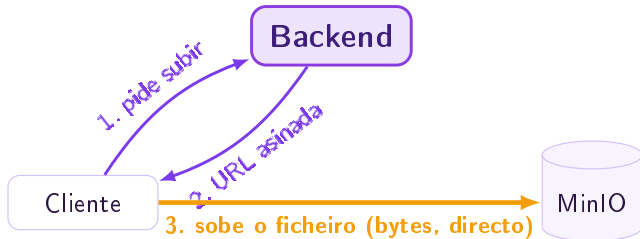
Por que

- O *access token* úsase constantemente → interézanos validalo **sen consultar a BD** (rápido e escalable). Como JWT asinado, **non se pode revogar**.
- O *refresh token* úsase **poucas veces** (só cando caduca o access), así que o custo de ir á BD é irrelevante.
- Por iso faise **opaco e gardado no servidor**: permite **invalidalo** (peche de sesión, cambio de contrasinal), imposíbel cun JWT.



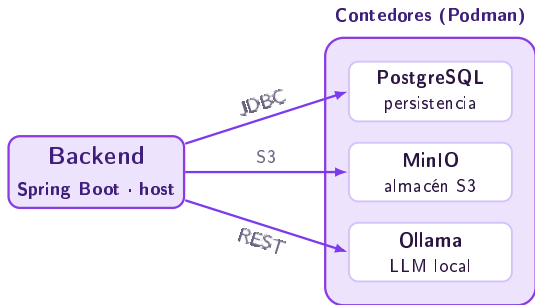
Por que SSE

- Só fai falta un fluxo **unidireccional** servidor→cliente.
- Máis simple que WebSockets (bidireccional), que aquí sobra.
- O servidor **persiste** a mensaxe e **difúndea** ao instante a todos os subscritores.



Por que

- Os **bytes non pasan polo backend**: el só emite unha URL temporal asinada.
- O cliente sobe/descarga **directamente** contra MinIO.
- Descarga o servidor da transferencia → mellor **escalabilidade**.



Por que

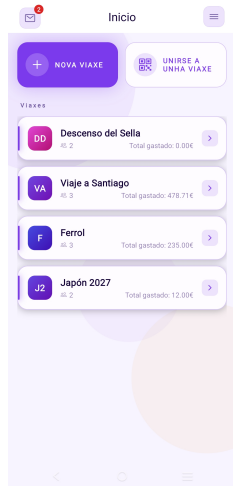
- As dependencias —**PostgreSQL**, **MinIO** e **Ollama**— corren cada unha no **seu propio contedor**; o backend execútase no host e conéctase a elas. Illamento e despregamento reproducible cun só *compose*.
- Comunícanse por **APIs ben definidas**: o almacén fala a **API S3** (estándar → migrable a AWS S3 **sen tocar código**); a IA, a **API REST de Ollama**, que serve calquera modelo local.
- Resultado: **sen vendor lock-in**. Execución local hoxe, portable mañá.

Aparencia

- Paleta **monocromática** centrada no **violeta**
- Vermello reservado para **accións destrutivas** (eliminar conta, saír da viaxe...)
- Tema **claro e escuro** seleccionable
- Tipografía nativa do sistema (coidada e rápida)

Accesibilidade

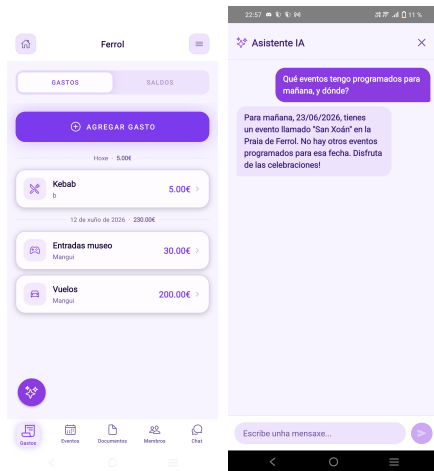
- Contraste **WCAG AA**
- Etiquetas e *roles* para lectores de pantalla
- Respecta o **movemento reducido** do sistema
- Área de pulsación ampliada ao mínimo recomendado



Percorrido rápido pola aplicación:

1. Unirse a unha viaxe escaneando un código QR
2. Rexistrar un gasto → **recálculo automático** de balances e liquidación suxerida
3. Crear un evento no calendario cun **mapa interactivo**
4. Asistente de **IA** respondendo en *streaming* co contexto da viaxe

► Reproducir vídeo da demostración



Probas automatizadas: 677

- **410** no backend (JUnit 5), con integración real contra PostgreSQL e MinIO vía **Testcontainers**
- **267** no frontend (Jest)
- Cobertura do backend $> 90\%$ de liñas

Avaliación de usabilidade

- Avaliación heurística (10 heurísticas de Nielsen)
- **15 usuarios reais**, protocolo de *pensar en voz alta*, ata a **saturación**

Melloras xa incorporadas:

- Acceso aos documentos do día dende o calendario
- Ordenación das viaxes por data de creación
- Mapa interactivo para escoller a localización dun evento

Os obxectivos cumpríronse na súa totalidade:

- Telaria **centraliza** a xestión integral dunha viaxe en grupo nunha soa app.
- A arquitectura **API-first** foi un acerto: mantén sincronizados frontend e backend.
- A **execución local do modelo de linguaxe** evita servizos externos de pago e preserva a privacidade.

Principal dificultade técnica

Integrar **Testcontainers** cun entorno de contedores sen *daemon* como **Podman**.

- Notificacións **push**.
- Preparación para **producción real**: externalizar segredos e habilitar HTTPS.
- Integrar as probas do **frontend** no *pipeline* de integración continua.
- Modo **sen conexión** con sincronización posterior, para destinos con conectividade limitada.

Grazas pola vosa atención

Quedo á vosa disposición para calquera pregunta.

Jorge Otero Pailos — Telaria — TFG · ETSE · USC